TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

BÀI TẬP LỚN KỸ THUẬT LẬP TRÌNH NÂNG CAO

ĐỀ TÀI: XÂY DỰNG PHẦN MỀM QUẢN LÝ SINH VIÊN SỬ DỤNG CẤU TRÚC MẢNG ĐỘNG VÀ GIAO DIỆN ĐỒ HỌA

GVHD: Ts. NGHIÊM VĂN TÍNH

Sinh viên thực hiện **HỨA THỊ THANH HIỀN**

Mssv **K225480106016**

Thái Nguyên 2026

TRƯỜNG ĐHKTCN	**CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM**
KHOA ĐIỆN TỬ	***Độc lập - Tự do - Hạnh phúc***

PHIẾU GIAO ĐỀ TÀI BÀI TẬP LỚN

BỘ MÔN : CÔNG NGHỆ THÔNG TIN

Sinh viên: Hứa Thị Thanh Hiền

MSSV: K225480106016 *Lớp: K58KTP*

Ngành: Kỹ thuật máy tính

GVHD: Ts.Nghiêm Văn Tính

Ngày giao đề: 24/12/2025 *Ngày hoàn thành: 13/03/2026*

1.Tên đề tài : Xây dựng phần mềm quản lý sinh viên sử dụng cấu trúc mảng động và giao diện đồ họa.

2. Nội dung thuyết minh tính toán:

Xây dựng phần mềm quản lý sinh viên sử dụng cấu trúc mảng động và giao diện đồ họa.tay cần đáp ứng các yêu cầu chức năng sau:

- Nghiên cứu cơ sở lý thuyết về cấu trúc dữ liệu mảng động và các thuật toán xử lý danh sách.
- Phân tích bài toán quản lý sinh viên, xác định các thông tin cần quản lý như: mã sinh viên, họ tên, ngày sinh, giới tính, lớp và điểm trung bình.
- Xây dựng các thuật toán thêm, sửa, xóa, tìm kiếm và sắp xếp sinh viên dựa trên cấu trúc mảng động.
- Đánh giá độ phức tạp của các thuật toán sử dụng trong chương trình.
- Thiết kế giao diện đồ họa thân thiện, hỗ trợ người dùng nhập liệu và thao tác quản lý sinh viên một cách trực quan.
- Kiểm tra, chạy thử chương trình với dữ liệu mẫu và đánh giá kết quả đạt được.

3. Các bản vẽ, chương trình và đồ thị:

- Phần mềm quản lý sinh viên
- Thuyết minh báo cáo + slide powerpoint

TRƯỞNG BỘ MÔN	GIÁO VIÊN HƯỚNG DẪN
(Ký và ghi rõ họ tên)	*(Ký và ghi rõ họ tên)*
	Ts. Nghiêm Văn Tính

LỜI CAM ĐOAN

Em xin cam đoan báo cáo với đề tài "Xây dựng phần mềm quản lý sinh viên sử dụng cấu trúc mảng động và giao diện đồ họa" là kết quả nghiên cứu và thực hiện của riêng em dưới sự hướng dẫn của TS. Nghiêm Văn Tính trong học phần Kỹ thuật lập trình nâng cao.

Các nội dung, số liệu và kết quả trình bày trong báo cáo là trung thực, không sao chép từ bất kỳ nguồn nào khác. Những tài liệu tham khảo (nếu có) đều đã được trích dẫn rõ ràng. Em xin hoàn toàn chịu trách nhiệm về nội dung của báo cáo này.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Hứa Thị Thanh Hiền

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, ngày… tháng… năm 2025

GIÁO VIÊN HƯỚNG DẪN CHẤM

MỤC LỤC

LỜI CAM ĐOAN 4
MỤC LỤC 6
DANH MỤC HÌNH ẢNH 9
DANH MỤC TỪ VIẾT TẮT 10
LỜI NÓI ĐẦU 11
LỜI CẢM ƠN 12
CHƯƠNG 1: CƠ SỞ DỮ LIỆU 13
1.1. Tổng quan về cấu trúc dữ liệu 13
1.1.1. Khái niệm cấu trúc dữ liệu 13
1.1.2. Phân loại cấu trúc dữ liệu 13
1.2. Cấu trúc dữ liệu mảng động 14
1.2.1. Định nghĩa và đặc điểm 14
1.2.2. Cơ chế cấp phát lại bộ nhớ (Resize) 15
1.2.3. Phân tích khấu hao (Amortized Analysis) 16
1.2.4. Lý do lựa chọn mảng động cho hệ thống 16
1.3. Lập trình hướng đối tượng (OOP) 17
1.4. Thuật toán sắp xếp 17
1.4.1. Khái niệm 17
1.4.2. QuickSort — Sắp xếp nhanh 18
1.4.3. BubbleSort — Sắp xếp nổi bọt 18
1.5. Thuật toán tìm kiếm 19
1.5.1. Tìm kiếm tuần tự (Linear Search) 19
1.5.2. Tìm kiếm nhị phân (Binary Search) 19

1.6. Thuật toán thống kê 19
1.7. Phân tích độ phức tạp thuật toán 20
Tóm tắt chương 1 21
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 22
2.1. Phân tích bài toán 22
2.1.1. Bối cảnh và nhu cầu thực tiễn 22
2.1.2. Xác định vấn đề cốt lõi 22
2.1.3. Mục tiêu của hệ thống 23
2.2. Phân tích yêu cầu hệ thống 23
2.2.1. Yêu cầu chức năng 23
2.2.2. Yêu cầu phi chức năng 24
2.3. Thiết kế kiến trúc hệ thống 25
2.4. Thiết kế mô hình dữ liệu 26
Tóm tắt chương 2 28
CHƯƠNG 3: THUẬT TOÁN SỬ DỤNG 29
3.1. Thuật toán trên mảng động 29
3.1.1. Thuật toán mở rộng mảng (_resize) 29
3.1.2. Thuật toán thêm phần tử vào cuối (push) 30
3.1.3. Thuật toán xóa phần tử tại vị trí (removeAt) 30
3.1.4. Thuật toán chèn phần tử tại vị trí (insertAt) 31
3.1.5. Thuật toán nhân bản mảng (clone) 32
3.2. Thuật toán sắp xếp QuickSort 33
3.2.1. Hàm đệ quy chính 33
3.2.2. Hàm phân hoạch (Partition) 33
3.2.3. Hàm so sánh sinh viên 34
3.3. Thuật toán sắp xếp BubbleSort 35

3.4. Thuật toán tìm kiếm tuần tự (Linear Search) 36
3.4.1. Tìm kiếm đơn giản 36
3.4.2. Tìm kiếm đa tiêu chí 37
3.5. Thuật toán tìm kiếm nhị phân (Binary Search) 38
3.6. Thuật toán thống kê Single-pass 39
3.7. Thuật toán phân loại học lực 41
Tóm tắt chương 3 42
CHƯƠNG 4: KẾT QUẢ VÀ BÀN LUẬN 43
4.1. Môi trường và công cụ phát triển 43
4.2. Giao diện và kết quả thực thi các chức năng 43
4.2.1. Phân hệ Tổng quan (Dashboard) 43
4.2.2. Phân hệ Thêm mới sinh viên 44
4.2.3. Phân hệ Quản lý danh sách sinh viên 45
4.2.4. Phân hệ Tìm kiếm 46
Tóm tắt chương 4 47
KẾT LUẬN 48
HƯỚNG PHÁT TRIỂN 49

DANH MỤC HÌNH ẢNH

Hình 2.4.1: Bảng minh họa quá trình mở rộng mảng động.....27

Hình 3.7.1: Thuật toán phân loại học lực.....41

Hình 4.2.1: Giao diện Tổng quan của hệ thống.....44

Hình 4.2.2: Giao diện biểu mẫu thêm mới sinh viên và tính năng kiểm tra dữ liệu....44

Hình 4.2.3: Giao diện quản lý danh sách sinh viên và công cụ sắp xếp động.....45

Hình 4.2.4: Kết quả thực thi chức năng tìm kiếm đa tiêu chí.....46

Hình 4.2.5: Giao diện thống kê tổng hợp và các biểu đồ trực quan.....47

DANH MỤC TỪ VIẾT TẮT

API: Application Programming Interface (Giao diện lập trình ứng dụng).

CRUD: Create, Read, Update, Delete (Thêm, Đọc, Cập nhật, Xóa).

CSS: Cascading Style Sheets (Ngôn ngữ tạo phong cách cho trang web).

ĐHKTCN: Đại học Kỹ thuật Công nghiệp.

GVHD: Giáo viên hướng dẫn.

HTML: HyperText Markup Language (Ngôn ngữ đánh dấu siêu văn bản).

JS: JavaScript.

JSON: JavaScript Object Notation (Định dạng dữ liệu văn bản).

MSSV: Mã số sinh viên.

OOP: Object-Oriented Programming (Lập trình hướng đối tượng).

PDF: Portable Document Format (Định dạng tài liệu di động).

SPA: Single Page Application (Ứng dụng trang web đơn).

SV: Sinh viên.

TB: Trung bình.

LỜI NÓI ĐẦU

Trong thời đại công nghệ thông tin phát triển mạnh mẽ như hiện nay, lập trình đóng vai trò quan trọng trong việc xây dựng và phát triển các hệ thống phần mềm phục vụ cho công tác quản lý, học tập và nghiên cứu. Việc nắm vững kiến thức lập trình không chỉ dừng lại ở việc viết được chương trình chạy đúng, mà còn đòi hỏi khả năng phân tích bài toán, lựa chọn cấu trúc dữ liệu phù hợp và xây dựng thuật toán hiệu quả. Chính vì vậy, học phần Kỹ thuật lập trình nâng cao có ý nghĩa quan trọng trong việc trang bị cho sinh viên những kiến thức và kỹ năng cần thiết để tiếp cận các bài toán lập trình ở mức độ cao hơn.

Thông qua học phần này, sinh viên được rèn luyện tư duy lập trình, khả năng tổ chức chương trình một cách khoa học, cũng như kỹ năng quản lý dữ liệu và bộ nhớ trong quá trình xây dựng phần mềm. Trên cơ sở đó, đề tài *"Xây dựng phần mềm quản lý sinh viên sử dụng cấu trúc mảng động và giao diện đồ họa"* được lựa chọn nhằm vận dụng các kiến thức đã học vào việc giải quyết một bài toán quản lý cụ thể, có tính ứng dụng thực tế.

Đề tài tập trung nghiên cứu và áp dụng cấu trúc dữ liệu mảng động để quản lý danh sách sinh viên, cho phép chương trình linh hoạt khi số lượng dữ liệu thay đổi. Bên cạnh đó, các thuật toán xử lý như thêm, xóa, tìm kiếm và sắp xếp dữ liệu sinh viên được xây dựng và đánh giá nhằm đảm bảo tính đúng đắn và hiệu quả của chương trình. Việc thiết kế giao diện đồ họa giúp người dùng dễ dàng tương tác với phần mềm, nâng cao tính trực quan và khả năng sử dụng.

Nội dung báo cáo bao gồm các phần chính như: cơ sở lý thuyết, phân tích bài toán, xây dựng thuật toán, cài đặt chương trình và đánh giá kết quả đạt được. Thông qua quá trình thực hiện đề tài, sinh viên có điều kiện củng cố kiến thức đã học, đồng thời tích lũy thêm kinh nghiệm trong việc phát triển một chương trình hoàn chỉnh. Tuy đã có nhiều cố gắng, nhưng do thời gian và trình độ còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy để báo cáo được hoàn thiện hơn.

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành đến TS. Nghiêm Văn Tính, giảng viên giảng dạy học phần Kỹ thuật lập trình nâng cao, đã tận tình hướng dẫn, truyền đạt kiến thức và góp ý trong suốt quá trình em thực hiện báo cáo này.

Em cũng xin cảm ơn Khoa và Nhà trường đã tạo điều kiện thuận lợi về môi trường học tập, giúp em có cơ hội vận dụng kiến thức lý thuyết vào thực hành. Mặc dù đã có nhiều cố gắng, song do thời gian và kinh nghiệm còn hạn chế, báo cáo không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy để bài báo cáo được hoàn thiện hơn.

Em xin chân thành cảm ơn!

CHƯƠNG 1: CƠ SỞ DỮ LIỆU

1.1. Tổng quan về cấu trúc dữ liệu

1.1.1. Khái niệm cấu trúc dữ liệu

Cấu trúc dữ liệu (Data Structure) là phương pháp tổ chức, lưu trữ và quản lý dữ liệu trong bộ nhớ máy tính nhằm tối ưu hóa quá trình truy cập, cập nhật và xử lý dữ liệu. Việc lựa chọn cấu trúc dữ liệu phù hợp ảnh hưởng trực tiếp đến hiệu năng, độ phức tạp thuật toán và khả năng mở rộng của hệ thống.

Một cấu trúc dữ liệu tốt cần đảm bảo các tiêu chí sau:

- Tối ưu hóa việc sử dụng bộ nhớ.
- Hỗ trợ các thao tác thêm, xóa, tìm kiếm, sắp xếp hiệu quả.
- Dễ mở rộng và bảo trì.
- Phù hợp với đặc thù bài toán.

Trong đề tài "Xây dựng phần mềm quản lý sinh viên sử dụng cấu trúc mảng động và giao diện đồ họa", bài toán đặt ra là phải xử lý danh sách sinh viên với số lượng có thể thay đổi theo thời gian, đồng thời đảm bảo truy cập nhanh và hỗ trợ nhiều thao tác sắp xếp, tìm kiếm. Mảng động là một phiên bản cải tiến của mảng tĩnh truyền thống, cho phép tự động mở rộng kích thước khi cần thiết, đồng thời vẫn giữ được ưu điểm truy cập trực tiếp O(1) của mảng.Vì vậy, cấu trúc mảng động được lựa chọn làm nền tảng dữ liệu chính cho hệ thống.

1.1.2. Phân loại cấu trúc dữ liệu

Việc phân loại giúp lựa chọn cấu trúc phù hợp với từng bài toán cụ thể, từ đó tối ưu hiệu năng xử lý và sử dụng bộ nhớ.

Dựa vào đặc điểm tổ chức, cấu trúc dữ liệu được chia thành hai nhóm chính sau:

Cấu trúc dữ liệu tuyến tính (Linear Data Structure)

Cấu trúc dữ liệu tuyến tính là kiểu cấu trúc trong đó các phần tử được sắp xếp theo một trình tự xác định. Mỗi phần tử (trừ phần tử đầu và cuối) sẽ có một phần tử đứng trước và một phần tử đứng sau.

Đặc điểm:

- Các phần tử được lưu trữ theo thứ tự tuần tự.
- Dễ dàng duyệt từ đầu đến cuối.
- Phù hợp với các bài toán xử lý danh sách.

Cấu trúc dữ liệu phi tuyến tính (Non-linear Data Structure)

Cấu trúc dữ liệu phi tuyến tính là kiểu cấu trúc trong đó các phần tử không được sắp xếp theo một trình tự đơn giản mà có thể liên kết với nhiều phần tử khác nhau. Mối quan hệ giữa các phần tử thường mang tính phân cấp hoặc mạng lưới.

Đặc điểm:

- Không tồn tại khái niệm "phần tử trước" và "phần tử sau" duy nhất.
- Có thể biểu diễn các quan hệ phức tạp giữa dữ liệu.
- Thường được sử dụng trong các bài toán phân cấp hoặc tìm kiếm nâng cao.

So với cấu trúc tuyến tính, cấu trúc phi tuyến tính có mức độ phức tạp cao hơn trong cài đặt nhưng lại rất hiệu quả trong các bài toán yêu cầu tổ chức dữ liệu theo nhiều mức hoặc nhiều mối quan hệ.

Việc phân loại cấu trúc dữ liệu thành tuyến tính và phi tuyến tính giúp định hướng lựa chọn giải pháp phù hợp cho từng bài toán cụ thể. Đối với hệ thống quản lý sinh viên trong đề tài này, dữ liệu chủ yếu được tổ chức dưới dạng danh sách, do đó cấu trúc dữ liệu tuyến tính - cụ thể là mảng động - là lựa chọn phù hợp.

1.2. Cấu trúc dữ liệu mảng động

1.2.1. Định nghĩa và đặc điểm

Mảng tĩnh (Static Array) là cấu trúc dữ liệu cơ bản nhất, trong đó các phần tử được lưu trữ liên tiếp nhau trong bộ nhớ với kích thước cố định ngay từ khi khai báo. Ví dụ, khi khai báo một mảng tĩnh có 100 phần tử, chương trình sẽ cấp phát đúng 100 ô nhớ liên tiếp, và kích thước này không thể thay đổi trong suốt quá trình chạy chương trình. Điều này dẫn đến hai vấn đề lớn: (1) Nếu số phần tử cần lưu trữ vượt quá 100,

chương trình sẽ bị tràn mảng (overflow); (2) Nếu chỉ sử dụng 10 ô nhớ, 90 ô nhớ còn lại bị lãng phí.

Mảng động (Dynamic Array) ra đời để khắc phục những hạn chế trên. Mảng động có khả năng tự động mở rộng kích thước khi mảng đầy. Cơ chế hoạt động cơ bản của mảng động như sau: khởi tạo với một dung lượng ban đầu nhỏ (ví dụ 10 phần tử), khi thêm phần tử mà mảng đã đầy (length ≥ capacity), hệ thống sẽ tự động tạo một mảng mới có dung lượng gấp đôi, sao chép toàn bộ dữ liệu từ mảng cũ sang mảng mới, rồi tiếp tục sử dụng mảng mới. Quá trình này được gọi là cấp phát lại (resize).

Ba thuộc tính quan trọng của mảng động:

- Vùng lưu trữ dữ liệu (data[]): Là khu vực bộ nhớ dùng để lưu các phần tử của mảng.
- Số lượng phần tử hiện tại (length): Là số phần tử đang thực sự được lưu trữ trong mảng tại thời điểm đang xét.
- Dung lượng tối đa hiện tại(capacity): Là số lượng phần tử tối đa mà mảng có thể chứa trước khi cần cấp phát lại bộ nhớ.

Sự phân biệt giữa length và capacity là đặc trưng quan trọng nhất của mảng động. Length cho biết bao nhiêu phần tử đang được sử dụng, còn capacity cho biết bao nhiêu phần tử có thể chứa trước khi cần resize. Điều này cho phép mảng động hoạt động hiệu quả: hầu hết các thao tác push đều có chi phí O(1) vì chỉ cần gán giá trị vào vị trí tiếp theo, chỉ thỉnh thoảng mới phải resize.

1.2.2. Cơ chế cấp phát lại bộ nhớ (Resize)

Để hiểu rõ hơn cách hoạt động của mảng động, chúng ta hãy xem xét ví dụ sau với mảng có dung lượng ban đầu là 4:

Bước 1: Khởi tạo mảng — capacity = 4, length = 0

[_][_][_][_]

Bước 2: Thêm 3 phần tử (A, B, C) — length = 3

[A][B][C][_]

Bước 3: Thêm phần tử thứ 4 (D) — length = 4 (ĐẦY!)

[A][B][C][D]

Bước 4: Thêm phần tử thứ 5 (E) → cần resize:

► Tạo mảng mới capacity = 8

► Copy dữ liệu cũ sang mảng mới

► Thêm phần tử E

[A][B][C][D][E][_][_][_]

1.2.3. Phân tích khấu hao (Amortized Analysis)

Một câu hỏi quan trọng khi sử dụng mảng động là: tại sao lại gấp đôi dung lượng khi resize? Câu trả lời nằm ở phân tích khấu hao (amortized analysis) — một phương pháp đánh giá chi phí trung bình của một chuỗi thao tác, thay vì chỉ xem xét chi phí của từng thao tác riêng lẻ.

Nếu mỗi lần resize chỉ tăng thêm 1 ô nhớ (tức capacity = capacity + 1), thì mỗi lần thêm phần tử khi mảng đầy đều phải resize, tức phải sao chép toàn bộ n phần tử → chi phí O(n) cho MỖI lần push. Tổng chi phí cho n lần push sẽ là $O(n^2)$, cực kỳ kém hiệu quả.

Ngược lại, nếu gấp đôi dung lượng (capacity = capacity × 2), thì sau k lần resize, tổng chi phí sao chép là: $n + n/2 + n/4 + n/8 + ... \approx 2n$. Chia đều cho n lần push, chi phí trung bình mỗi lần push chỉ là $O(2n/n) = O(1)$. Đây chính là nguyên lý phân tích khấu hao: mặc dù một vài thao tác riêng lẻ có thể rất tốn kém (O(n) khi resize), nhưng tính trung bình trên toàn bộ chuỗi thao tác, chi phí mỗi thao tác chỉ là O(1).

1.2.4. Lý do lựa chọn mảng động cho hệ thống

Hệ thống quản lý sinh viên yêu cầu:

- Truy cập dữ liệu nhanh theo chỉ số.
- Sắp xếp và tìm kiếm thường xuyên.
- Hiển thị danh sách dạng bảng.

Mảng động đáp ứng tốt các yêu cầu trên, đảm bảo hiệu năng xử lý và dễ dàng tích hợp với các thuật toán sắp xếp như thuật toán sắp xếp nhanh trong quá trình xây dựng chương trình.

1.3. Lập trình hướng đối tượng (OOP)

Lập trình hướng đối tượng là một phương pháp lập trình dựa trên khái niệm "đối tượng" (object). Mỗi đối tượng chứa dữ liệu dưới dạng các thuộc tính và hành vi dưới dạng các phương thức. OOP giúp tổ chức mã nguồn rõ ràng, dễ bảo trì và mở rộng thông qua bốn nguyên lý cơ bản: đóng gói, kế thừa đa hình và trừu tượng hóa.

Trong hệ thống quản lý sinh viên này, em áp dụng OOP với các lớp chính sau:

- Lớp Student (SinhVien.js): Đại diện cho một sinh viên với các thuộc tính như mã sinh viên, họ tên, ngày sinh, giới tính, lớp, email, số điện thoại, địa chỉ và điểm số các môn. Lớp này cũng chứa các phương thức tính điểm trung bình, xếp loại học lực và kiểm tra dữ liệu hợp lệ.
- Lớp DynamicArray (MangDong.js): Triển khai cấu trúc dữ liệu mảng động với các thao tác push, pop, get, set, insertAt, removeAt, find, filter, sort, binarySearch và clone.
- Lớp StudentManager (QuanLySinhVien.js): Quản lý danh sách sinh viên sử dụng mảng động, cung cấp các chức năng CRUD (Create, Read, Update, Delete), tìm kiếm, sắp xếp và thống kê.
- Lớp UIController (DieuKhienGiaoDien.js): Điều khiển giao diện người dùng, xử lý sự kiện, hiển thị bảng biểu và biểu đồ thống kê.
- Lớp Application (UngDung.js): Điểm khởi đầu của ứng dụng, kết nối tất cả các module lại với nhau.

1.4. Thuật toán sắp xếp

1.4.1. Khái niệm

Sắp xếp (Sorting) là quá trình sắp xếp lại các phần tử trong một tập dữ liệu theo một thứ tự nhất định (tăng dần hoặc giảm dần) dựa trên một khóa so sánh (sort key). Sắp xếp là một trong những bài toán cơ bản và quan trọng nhất trong khoa học máy tính, bởi

vì nhiều thuật toán khác (như tìm kiếm nhị phân) yêu cầu dữ liệu đầu vào phải được sắp xếp trước.

1.4.2. QuickSort — Sắp xếp nhanh

QuickSort là thuật toán sắp xếp dựa trên chiến lược "chia để trị" (Divide and Conquer), được phát minh bởi Tony Hoare vào năm 1960. Ý tưởng cốt lõi của QuickSort là: chọn một phần tử làm "chốt" (pivot), sau đó phân hoạch mảng thành hai phần — phần bên trái chứa các phần tử nhỏ hơn hoặc bằng pivot, phần bên phải chứa các phần tử lớn hơn pivot. Quá trình này được lặp lại đệ quy cho từng phần cho đến khi mảng được sắp xếp hoàn toàn.

Các bước thực hiện QuickSort:

Bước 1: Chọn phần tử chốt (pivot). Trong triển khai của chúng tôi, chúng tôi chọn phần tử cuối cùng làm pivot.
Bước 2: Phân hoạch (Partition) — duyệt mảng từ trái sang phải, đưa các phần tử nhỏ hơn pivot về bên trái, các phần tử lớn hơn pivot về bên phải.
Bước 3: Đặt pivot vào đúng vị trí cuối cùng của nó trong mảng đã sắp xếp.
Bước 4: Đệ quy sắp xếp nửa trái (từ đầu đến pivot - 1).
Bước 5: Đệ quy sắp xếp nửa phải (từ pivot + 1 đến cuối).

Độ phức tạp thời gian:

- Trường hợp tốt nhất và trung bình: O(n log n) — khi pivot chia đều mảng thành hai nửa xấp xỉ bằng nhau.
- Trường hợp xấu nhất: $O(n^2)$ — khi mảng đã sắp xếp sẵn và luôn chọn phần tử nhỏ nhất/lớn nhất làm pivot.
- Bộ nhớ phụ: O(log n) — cho ngăn xếp đệ quy.

1.4.3. BubbleSort — Sắp xếp nổi bọt

BubbleSort là một trong những thuật toán sắp xếp đơn giản nhất. Ý tưởng của thuật toán là duyệt mảng nhiều lần, mỗi lần so sánh hai phần tử liền kề và hoán đổi nếu chúng không đúng thứ tự. Sau mỗi vòng lặp, phần tử lớn nhất (chưa đúng vị trí) sẽ "nổi" lên cuối mảng, giống như bong bóng nổi lên mặt nước — đó là lý do tên gọi "sắp xếp nổi bọt".

Độ phức tạp thời gian:

- Trường hợp tốt nhất: O(n) — khi mảng đã sắp xếp sẵn (có cờ dừng sớm).
- Trường hợp trung bình và xấu nhất: $O(n^2)$.
- Bộ nhớ phụ: O(1) — sắp xếp tại chỗ.

1.5. Thuật toán tìm kiếm

1.5.1. Tìm kiếm tuần tự (Linear Search)

Tìm kiếm tuần tự là thuật toán tìm kiếm cơ bản nhất: duyệt lần lượt từng phần tử trong mảng, so sánh từng phần tử với giá trị cần tìm. Nếu tìm thấy phần tử phù hợp, thuật toán trả về vị trí (index) của phần tử đó; nếu duyệt hết mảng mà không tìm thấy, thuật toán trả về -1 để báo không tìm thấy.

- Ưu điểm của tìm kiếm tuần tự là đơn giản, dễ cài đặt, và đặc biệt không yêu cầu mảng phải được sắp xếp trước.
- Nhược điểm là hiệu suất thấp khi mảng có kích thước lớn (O(n) cho mọi trường hợp).

1.5.2. Tìm kiếm nhị phân (Binary Search)

Tìm kiếm nhị phân là thuật toán tìm kiếm hiệu quả cao, hoạt động trên mảng ĐÃ ĐƯỢC SẮP XẾP. Ý tưởng của thuật toán là chia đôi không gian tìm kiếm ở mỗi bước: so sánh phần tử ở giữa mảng với giá trị cần tìm, nếu bằng nhau thì tìm thấy, nếu nhỏ hơn thì tìm trong nửa phải, nếu lớn hơn thì tìm trong nửa trái. Quá trình lặp lại cho đến khi tìm thấy hoặc đoạn tìm kiếm rỗng.

So sánh hiệu suất với tìm kiếm tuần tự: với n = 1.000 phần tử, tìm kiếm tuần tự cần tối đa 1.000 bước, trong khi tìm kiếm nhị phân chỉ cần tối đa 10 bước ($\log_2(1000) \approx 10$). Với n = 1.000.000 phần tử, tìm kiếm tuần tự cần tối đa 1.000.000 bước, còn tìm kiếm nhị phân chỉ cần khoảng 20 bước. Sự khác biệt là cực kỳ lớn.

1.6. Thuật toán thống kê

Thuật toán thống kê trong hệ thống được thiết kế theo nguyên lý "duyệt một lần duy nhất" (single-pass), nghĩa là chỉ cần duyệt qua mảng dữ liệu một lần nhưng thu thập được đồng thời nhiều chỉ số thống kê. Trong mỗi bước duyệt, thuật toán cập nhật song

song: tổng điểm (để tính trung bình), điểm cao nhất (max), điểm thấp nhất (min), số lượng sinh viên theo từng xếp loại, và số lượng sinh viên theo từng lớp.

Ưu điểm của phương pháp single-pass: hiệu quả gấp nhiều lần so với việc duyệt riêng lẻ cho từng chỉ số. Nếu cần tính 5 chỉ số khác nhau và mỗi chỉ số duyệt 1 lần → phải duyệt 5n phần tử. Với single-pass, chỉ cần duyệt n phần tử nhưng vẫn thu thập được tất cả 5 chỉ số. Thuật toán đếm và phân nhóm sử dụng bảng băm (hash map/object) để đếm số phần tử thuộc mỗi nhóm, với chi phí O(n) thời gian và O(k) bộ nhớ (k là số nhóm).

1.7. Phân tích độ phức tạp thuật toán

Độ phức tạp thuật toán (Algorithm Complexity) là thước đo đánh giá hiệu suất của thuật toán, thường được biểu diễn bằng ký hiệu Big-O. Ký hiệu Big-O mô tả tốc độ tăng trưởng của thời gian thực thi (hoặc bộ nhớ sử dụng) theo kích thước đầu vào n. Các lớp độ phức tạp phổ biến từ tốt đến tệ: O(1) — hằng số, O(log n) — logarit, O(n) — tuyến tính, O(n log n) — tuyến tính-logarit, $O(n^2)$ — bình phương, $O(2^n)$ — mũ.

Ký hiệu Big-O	**Tên gọi**	**Ví dụ**
O(1)	Hằng số	Truy cập mảng theo chỉ số
O(log n)	Logarit	Tìm kiếm nhị phân
O(n)	Tuyến tính	Tìm kiếm tuần tự, duyệt mảng
O(n log n)	Tuyến tính-logarit	QuickSort, MergeSort
$O(n^2)$	Bình phương	BubbleSort, hai vòng lặp lồng nhau
$O(2^n)$	Mũ	Vét cạn tất cả tập con

Hình 1.7.1: phân tích độ phức tạp của thuật toán

Tóm tắt chương 1

Chương 1 trình bày những kiến thức cơ bản dùng để xây dựng phần mềm. Nội dung chính là giải thích về mảng động – một loại mảng có thể tự mở rộng khi đầy. Chương cũng nói rõ cách mảng tự tăng kích thước, vì sao thao tác thêm phần tử thường nhanh (trung bình O(1)). Ngoài ra, chương giới thiệu lập trình hướng đối tượng và các thuật toán được dùng trong hệ thống như sắp xếp (QuickSort, BubbleSort), tìm kiếm (tuyến tính, nhị phân), thống kê và phân tích độ phức tạp Big-O. Đây là phần nền tảng lý thuyết cho toàn bộ đề tài.

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích bài toán

2.1.1. Bối cảnh và nhu cầu thực tiễn

Trong môi trường giáo dục đại học, việc quản lý thông tin sinh viên là một hoạt động mang tính thường xuyên và có vai trò quan trọng trong công tác đào tạo. Mỗi sinh viên có nhiều thông tin cần lưu trữ và theo dõi như mã số sinh viên, họ tên, lớp học, thông tin liên hệ và kết quả học tập. Khi số lượng sinh viên tăng lên theo từng khóa, phương pháp quản lý thủ công hoặc sử dụng bảng tính rời rạc dần bộc lộ nhiều hạn chế như khó mở rộng, dễ sai sót và khó thực hiện thống kê tổng hợp.

Từ yêu cầu thực tiễn đó, việc xây dựng một hệ thống quản lý sinh viên có tổ chức, có cấu trúc dữ liệu rõ ràng và hỗ trợ các thao tác xử lý tự động là cần thiết. Bên cạnh mục tiêu ứng dụng, đề tài còn nhằm phục vụ yêu cầu học thuật của học phần Lập trình nâng cao, trong đó sinh viên phải vận dụng cấu trúc dữ liệu và thuật toán để giải quyết một bài toán cụ thể.

Do đó, hệ thống được xây dựng không chỉ để quản lý thông tin sinh viên mà còn để thể hiện rõ quá trình thiết kế cấu trúc mảng động, phân tích độ phức tạp của thuật toán tìm kiếm và sắp xếp, cũng như tổ chức chương trình theo mô hình hướng đối tượng.

2.1.2. Xác định vấn đề cốt lõi

Vấn đề trung tâm của hệ thống là quản lý một tập hợp sinh viên có kích thước thay đổi theo thời gian. Danh sách sinh viên không cố định về số lượng vì có thể phát sinh thêm hoặc xóa bớt trong quá trình sử dụng. Điều này đặt ra yêu cầu về một cấu trúc dữ liệu linh hoạt, có khả năng mở rộng khi cần thiết.

Nếu sử dụng mảng tĩnh với kích thước cố định, hệ thống sẽ gặp khó khăn khi số lượng sinh viên vượt quá giới hạn ban đầu. Vì vậy, cấu trúc mảng động được lựa chọn làm nền tảng lưu trữ chính. Mảng động cho phép cấp phát bộ nhớ ban đầu với dung lượng nhất định và tự động mở rộng khi số phần tử đạt giới hạn.

Cơ chế mở rộng được thực hiện bằng cách tạo một vùng nhớ mới có dung lượng lớn hơn, sao chép toàn bộ dữ liệu từ vùng nhớ cũ sang vùng nhớ mới, sau đó giải phóng vùng nhớ cũ. Thông thường, dung lượng mới sẽ được tăng gấp đôi so với dung lượng hiện tại để giảm số lần cấp phát lại. Nhờ cơ chế này, thao tác thêm phần tử có độ phức tạp khấu hao trung bình là O(1), mặc dù tại thời điểm mở rộng có thể phát sinh chi phí O(n).

Bên cạnh vấn đề lưu trữ, hệ thống còn cần hỗ trợ các thao tác xử lý dữ liệu như tìm kiếm, sắp xếp và thống kê. Do đó, việc lựa chọn thuật toán phù hợp đóng vai trò quan trọng trong thiết kế tổng thể.

2.1.3. Mục tiêu của hệ thống

Hệ thống được xây dựng nhằm đạt được ba nhóm mục tiêu chính.

Mục tiêu học thuật là thể hiện rõ quá trình xây dựng cấu trúc mảng động, cơ chế quản lý bộ nhớ và phân tích độ phức tạp của từng thao tác. Hệ thống phải chứng minh được sự hiểu biết về bản chất của các thuật toán và cách áp dụng vào bài toán thực tế.

Mục tiêu kỹ thuật là tổ chức chương trình theo nguyên tắc lập trình hướng đối tượng. Các lớp phải được thiết kế rõ ràng, có tính đóng gói và phân chia trách nhiệm cụ thể. Mã nguồn phải được tổ chức khoa học, tránh trộn lẫn logic xử lý với giao diện.

Mục tiêu ứng dụng là tạo ra một phần mềm có thể sử dụng thực tế để quản lý danh sách sinh viên, thực hiện tìm kiếm nhanh chóng, sắp xếp linh hoạt và cung cấp các thống kê tổng hợp phục vụ theo dõi kết quả học tập.

2.2. Phân tích yêu cầu hệ thống

2.2.1. Yêu cầu chức năng

Hệ thống phải đảm bảo đầy đủ các chức năng cơ bản của một phần mềm quản lý sinh viên.

Hệ thống phải cho phép thêm mới sinh viên. Khi người dùng nhập thông tin, hệ thống thực hiện kiểm tra tính hợp lệ của dữ liệu như không để trống các trường bắt buộc, điểm số nằm trong khoảng cho phép và mã sinh viên không trùng lặp. Sau khi kiểm tra thành công, đối tượng sinh viên được thêm vào mảng động.

Hệ thống phải cho phép chỉnh sửa thông tin sinh viên. Khi thực hiện thao tác cập nhật, hệ thống xác định vị trí của sinh viên trong danh sách, sau đó thay đổi các thuộc tính tương ứng và lưu lại trạng thái mới.

thống phải hỗ trợ xóa sinh viên khỏi danh sách. Khi xóa một phần tử trong mảng, các phần tử phía sau sẽ được dịch chuyển lên để đảm bảo tính liên tục của dữ liệu. Do đặc điểm của mảng, thao tác này có độ phức tạp O(n).

Hệ thống phải hỗ trợ tìm kiếm sinh viên theo nhiều tiêu chí như mã sinh viên, họ tên hoặc lớp học. Do danh sách có thể được sắp xếp theo các tiêu chí khác nhau và không cố định thứ tự, phương pháp tìm kiếm tuần tự được sử dụng để đảm bảo tính linh hoạt.

Hệ thống phải hỗ trợ sắp xếp danh sách sinh viên theo các tiêu chí như mã sinh viên hoặc điểm trung bình. Thuật toán QuickSort được lựa chọn vì có độ phức tạp trung bình O(n log n) và phù hợp với cấu trúc mảng liên tục trong bộ nhớ.

Hệ thống phải cung cấp chức năng thống kê như tính điểm trung bình chung của toàn bộ sinh viên, xác định sinh viên có điểm cao nhất và thấp nhất, đồng thời phân loại học lực. Các chỉ số này được tính trong một lần duyệt mảng để tối ưu hiệu năng xử lý.

2.2.2. Yêu cầu phi chức năng

Bên cạnh yêu cầu chức năng, hệ thống cần đáp ứng các yêu cầu phi chức năng để đảm bảo tính ổn định và chất lượng.

Về hiệu năng, hệ thống phải xử lý tốt với quy mô vài trăm đến một nghìn sinh viên mà không gây chậm trễ đáng kể. Các thao tác tìm kiếm và sắp xếp phải có thời gian phản hồi hợp lý.

Về tính chính xác, hệ thống phải kiểm tra dữ liệu đầu vào trước khi lưu trữ. Các điều kiện ràng buộc như không trùng mã sinh viên và giới hạn điểm số phải được đảm bảo.

Về khả năng mở rộng, cấu trúc chương trình phải cho phép bổ sung thêm môn học hoặc chuyển sang lưu trữ dữ liệu bằng cơ sở dữ liệu thực tế mà không cần thay đổi toàn bộ hệ thống.

Về khả năng bảo trì, mã nguồn phải được tổ chức rõ ràng theo từng lớp, đảm bảo mỗi lớp có một trách nhiệm riêng biệt.

2.3. Thiết kế kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình phân lớp nhằm đảm bảo tính tổ chức, dễ bảo trì và thuận lợi cho việc mở rộng sau này.

Kiến trúc gồm ba tầng chính: tầng giao diện, tầng xử lý nghiệp vụ và tầng dữ liệu. Mỗi tầng đảm nhận một vai trò riêng biệt và giao tiếp với nhau theo thứ tự từ trên xuống dưới.

Tầng giao diện

- Tầng giao diện là nơi người dùng trực tiếp thao tác với hệ thống. Thành phần này hiển thị danh sách sinh viên, biểu mẫu nhập liệu và kết quả thống kê.
- Khi người dùng thực hiện thao tác như thêm, sửa, xóa hoặc tìm kiếm, giao diện sẽ tiếp nhận dữ liệu và gửi yêu cầu xuống tầng xử lý nghiệp vụ. Sau khi nhận kết quả xử lý, giao diện cập nhật lại bảng dữ liệu hoặc hiển thị thông báo tương ứng.
- Tầng này không chứa thuật toán xử lý phức tạp mà chỉ đóng vai trò trung gian giữa người dùng và hệ thống.

Tầng xử lý nghiệp vụ

- Tầng xử lý nghiệp vụ là trung tâm của hệ thống. Đây là nơi triển khai các thuật toán và điều phối dữ liệu.
- Các chức năng như thêm sinh viên, cập nhật thông tin, xóa phần tử, tìm kiếm và sắp xếp đều được thực hiện tại tầng này. Khi nhận yêu cầu từ giao diện, tầng nghiệp vụ sẽ kiểm tra dữ liệu, tạo đối tượng sinh viên và thực hiện thao tác trên mảng động.
- Thuật toán tìm kiếm tuần tự được sử dụng để duyệt danh sách theo tiêu chí lựa chọn. Thuật toán QuickSort được áp dụng khi cần sắp xếp theo mã sinh viên hoặc điểm trung bình. Ngoài ra, các phép thống kê như tính điểm trung bình chung và phân loại học lực cũng được xử lý tại đây.
- Việc tập trung toàn bộ logic xử lý tại một tầng riêng giúp hệ thống rõ ràng và dễ kiểm soát.

Tầng dữ liệu

- Tầng dữ liệu chịu trách nhiệm lưu trữ và quản lý cấu trúc dữ liệu của hệ thống.
- Lớp Student đại diện cho một sinh viên, bao gồm thông tin cá nhân và điểm số. Lớp này cung cấp phương thức tính điểm trung bình và xác định xếp loại học lực.
- Lớp DynamicArray quản lý danh sách sinh viên trong bộ nhớ. Cấu trúc này gồm vùng lưu trữ dữ liệu, số phần tử hiện tại và dung lượng tối đa. Khi mảng đầy, hệ thống sẽ cấp phát bộ nhớ mới với kích thước lớn hơn và sao chép dữ liệu sang vùng nhớ mới.
- Nhờ sử dụng mảng liên tục trong bộ nhớ, thao tác truy cập theo chỉ số có độ phức tạp O(1), phù hợp cho các thuật toán xử lý dữ liệu.

2.4. Thiết kế mô hình dữ liệu

Mô hình dữ liệu của hệ thống được xây dựng theo hướng đối tượng, trong đó mỗi sinh viên được biểu diễn dưới dạng một đối tượng độc lập. Toàn bộ danh sách sinh viên được lưu trữ trong cấu trúc mảng động.

Thiết kế này đảm bảo dữ liệu được tổ chức rõ ràng, dễ quản lý và thuận tiện cho việc xử lý thuật toán.

Thực thể Sinh viên

Sinh viên là thực thể trung tâm của hệ thống.

- Mỗi sinh viên bao gồm các nhóm thông tin chính như thông tin cá nhân, thông tin học tập và thông tin liên hệ.
- Thông tin cá nhân gồm mã sinh viên, họ tên, ngày sinh và giới tính.
  Thông tin học tập gồm lớp học và các điểm thành phần.
  Thông tin liên hệ gồm email, số điện thoại và địa chỉ.
- Bên cạnh các thuộc tính dữ liệu, đối tượng sinh viên còn có các phương thức xử lý như tính điểm trung bình và xác định xếp loại học lực.
- Điểm trung bình được tính theo công thức trung bình cộng của các môn học. Dựa trên kết quả này, hệ thống phân loại học lực theo các mức quy định.

- Việc đặt các phương thức xử lý bên trong lớp Student giúp đảm bảo tính đóng gói và đúng nguyên tắc lập trình hướng đối tượng.

Cấu trúc mảng động

Danh sách sinh viên không có kích thước cố định nên được lưu trữ trong mảng động.

- Cấu trúc mảng động gồm ba thành phần chính: vùng nhớ lưu trữ dữ liệu, số lượng phần tử hiện tại và dung lượng tối đa của mảng.
- Khi số lượng sinh viên đạt tới giới hạn dung lượng, hệ thống sẽ thực hiện cấp phát bộ nhớ mới với kích thước lớn hơn. Toàn bộ dữ liệu sẽ được sao chép sang vùng nhớ mới và vùng nhớ cũ sẽ được giải phóng.
- Cơ chế này giúp hệ thống mở rộng linh hoạt mà không cần xác định trước số lượng sinh viên tối đa. Đồng thời, thao tác truy cập phần tử theo chỉ số vẫn có độ phức tạp O(1).

Bảng minh họa quá trình mở rộng mảng động

Giai đoạn	**Capacity**	**Size**	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Ban đầu (chưa đầy)	4	3	SV1	SV2	SV3	(Trống)				
Khi đầy	4	4	SV1	SV2	SV3	SV4				
Sau khi resize (cấp phát mới)	8	4	SV1	SV2	SV3	SV4	(Trống)	(Trống)	(Trống)	(Trống)
Sau khi thêm SV5	8	5	SV1	SV2	SV3	SV4	SV5	(Trống)	(Trống)	(Trống)

Hình 2.4.1: Bảng minh họa quá trình mở rộng mảng động

Quan hệ giữa các thành phần

- Lớp DynamicArray chịu trách nhiệm lưu trữ các đối tượng Student.

- Tầng xử lý nghiệp vụ không thao tác trực tiếp với dữ liệu thô mà thông qua các phương thức của mảng động. Điều này giúp đảm bảo tính an toàn dữ liệu và tránh truy cập trái phép vào cấu trúc bên trong.
- Nhờ mô hình dữ liệu được thiết kế rõ ràng, hệ thống có thể dễ dàng bổ sung thêm thuộc tính hoặc mở rộng chức năng mà không ảnh hưởng đến toàn bộ chương trình.

Tóm tắt chương 2

Chương 2 tập trung phân tích bài toán quản lý sinh viên và thiết kế hệ thống. Trước hết, chương xác định vấn đề là cần quản lý danh sách sinh viên có số lượng thay đổi theo thời gian. Từ đó, mảng động được chọn để lưu trữ dữ liệu. Chương cũng nêu rõ các chức năng chính của hệ thống như thêm, sửa, xóa, tìm kiếm, sắp xếp và thống kê sinh viên. Hệ thống được thiết kế theo 3 phần: giao diện, xử lý nghiệp vụ và dữ liệu. Dữ liệu được tổ chức xoay quanh lớp Student và lớp DynamicArray, giúp chương trình rõ ràng và dễ mở rộng.

CHƯƠNG 3: THUẬT TOÁN SỬ DỤNG

Chương này trình bày chi tiết các thuật toán được sử dụng trong hệ thống quản lý sinh viên. Mỗi thuật toán được phân tích kỹ lưỡng về nguyên lý hoạt động, mã giả (pseudocode), ví dụ minh họa cụ thể, và đánh giá độ phức tạp.

3.1. Thuật toán trên mảng động

3.1.1. Thuật toán mở rộng mảng (_resize)

Đây là thuật toán quan trọng nhất và đặc trưng nhất của mảng động. Khi số phần tử hiện tại (length) bằng dung lượng tối đa (capacity), mảng cần được mở rộng để chứa thêm phần tử mới.

Mã giả:

```
FUNCTION _resize():
   dungLuongMoi = capacity × 2
   mangMoi = TẠO MẢNG MỚI kích thước dungLuongMoi
   FOR i = 0 TO length - 1:
      mangMoi[i] = data[i]    // Copy từng phần tử
   data = mangMoi
   capacity = dungLuongMoi
```

Ví dụ minh họa:

Giả sử mảng đang chứa 4 sinh viên {SV001, SV002, SV003, SV004} với capacity = 4. Khi thêm sinh viên thứ 5 (SV005), hàm push() phát hiện length (4) ≥ capacity (4) → gọi _resize():

Trước resize:

```
  data = [SV001][SV002][SV003][SV004]  (length=4, capacity=4)


  Bước 1: dungLuongMoi = 4 × 2 = 8
  Bước 2: Tạo mangMoi có 8 ô
  Bước 3: Copy 4 phần tử sang mangMoi
```

Sau resize:

data = [SV001][SV002][SV003][SV004][_][_][_][_] (length=4, capacity=8)

Bây giờ thêm SV005:

data = [SV001][SV002][SV003][SV004][SV005][_][_][_] (length=5, capacity=8)

Độ phức tạp: O(n) — cần copy n phần tử sang mảng mới.

3.1.2. Thuật toán thêm phần tử vào cuối (push)

Thao tác push là thao tác được sử dụng thường xuyên nhất trong hệ thống — mỗi lần thêm sinh viên mới đều gọi push. Thuật toán kiểm tra xem mảng có đầy không, nếu đầy thì mở rộng, sau đó gán phần tử vào vị trí length và tăng length lên 1.

Mã giả:

```
FUNCTION push(phanTu):
   IF length >= capacity:
      _resize()            // Mở rộng nếu đầy
   data[length] = phanTu      // Gán vào vị trí tiếp theo
   length = length + 1        // Tăng số phần tử
   RETURN length
```

Độ phức tạp: O(1) trung bình (phân tích khấu hao), O(n) trường hợp xấu nhất (khi resize).

3.1.3. Thuật toán xóa phần tử tại vị trí (removeAt)

Khi xóa sinh viên khỏi danh sách, thuật toán removeAt được sử dụng. Thao tác này cần dịch chuyển tất cả các phần tử sau vị trí xóa lên trước 1 ô để lấp đầy khoảng trống.

Mã giả:

```
FUNCTION removeAt(viTri):
   IF viTri < 0 OR viTri >= length:
      RETURN undefined       // Vị trí không hợp lệ
   phanTuDaXoa = data[viTri]  // Lưu phần tử cần xóa
   FOR i FROM viTri TO length - 2:
      data[i] = data[i + 1]  // Dịch phần tử lên trước
   data[length - 1] = undefined
   length = length - 1
   RETURN phanTuDaXoa
```

Ví dụ minh họa:

Xóa sinh viên tại vị trí index = 1 (SV002) trong mảng [SV001, SV002, SV003, SV004]:

Trước: [SV001][SV002][SV003][SV004] (length=4)
            ↑ xóa vị trí 1

Dịch: data[1] = data[2] → [SV001][SV003][SV003][SV004]
      data[2] = data[3] → [SV001][SV003][SV004][SV004]
      data[3] = undefined

Sau: [SV001][SV003][SV004][_] (length=3)

Độ phức tạp: O(n) — xóa ở đầu phải dịch n-1 phần tử, xóa ở cuối là O(1).

3.1.4. Thuật toán chèn phần tử tại vị trí (insertAt)

Thuật toán chèn phần tử tại một vị trí bất kỳ trong mảng. Các phần tử từ vị trí chèn đến cuối phải được dịch ra sau 1 ô để tạo chỗ trống. Lưu ý quan trọng: phải dịch TỪ CUỐI về đầu để tránh ghi đè dữ liệu.

Mã giả:

```
FUNCTION insertAt(viTri, phanTu):
   IF viTri < 0 OR viTri > length: RETURN false
   IF length >= capacity: _resize()
   FOR i FROM length DOWNTO viTri + 1:
      data[i] = data[i-1]    // Dịch ra sau (từ cuối!)
   data[viTri] = phanTu        // Đặt phần tử mới
   length = length + 1
   RETURN true
```

Ví dụ minh họa:

Chèn sinh viên "SV_NEW" tại vị trí 1 trong mảng [SV001, SV003, SV004]:

Trước: [SV001][SV003][SV004][_] (length=3, capacity=4)

Dịch: data[3] = data[2] → [SV001][SV003][SV004][SV004]
      data[2] = data[1] → [SV001][SV003][SV003][SV004]

Chèn: data[1] = SV_NEW → [SV001][SV_NEW][SV003][SV004]

Sau: [SV001][SV_NEW][SV003][SV004] (length=4)

Độ phức tạp: O(n) — phải dịch các phần tử.

3.1.5. Thuật toán nhân bản mảng (clone)

Thuật toán nhân bản tạo một bản sao hoàn toàn mới của mảng, giúp sắp xếp hiển thị mà không ảnh hưởng đến thứ tự gốc. Trong hệ thống, mỗi khi sắp xếp danh sách sinh viên, mảng gốc được clone trước, sau đó mới sắp xếp bản sao.

Mã giả:

```
FUNCTION clone():
   banSao = TẠO DynamicArray MỚI (capacity = this.capacity)
   FOR i = 0 TO length - 1:
```

```
    banSao.push(data[i])   // Copy từng phần tử
  RETURN banSao
```

Độ phức tạp: O(n) — copy n phần tử.

3.2. Thuật toán sắp xếp QuickSort

QuickSort là thuật toán sắp xếp chính được sử dụng trong hệ thống để sắp xếp danh sách sinh viên theo các tiêu chí: mã sinh viên, tên, điểm trung bình hoặc lớp. Thuật toán hoạt động theo chiến lược "chia để trị" (Divide and Conquer) với ba bước chính: chọn chốt (pivot), phân hoạch (partition), và đệ quy sắp xếp hai phần.

3.2.1. Hàm đệ quy chính

Mã giả:

```
FUNCTION quickSort(mang, viTriDau, viTriCuoi, hamSoSanh):
  IF viTriDau >= viTriCuoi:
    RETURN                // Điều kiện dừng
  viTriChot = phanHoach(mang, viTriDau, viTriCuoi, hamSoSanh)
  quickSort(mang, viTriDau, viTriChot - 1, hamSoSanh)     // Nửa trái
  quickSort(mang, viTriChot + 1, viTriCuoi, hamSoSanh)    // Nửa phải
```

3.2.2. Hàm phân hoạch (Partition)

Hàm phân hoạch là trái tim của QuickSort. Nó chọn phần tử cuối cùng làm chốt (pivot), sau đó duyệt mảng từ trái sang phải, đưa các phần tử nhỏ hơn hoặc bằng pivot về bên trái. Biến i đóng vai trò là "biên" của vùng chứa các phần tử ≤ pivot.

Mã giả:

```
FUNCTION phanHoach(mang, viTriDau, viTriCuoi, hamSoSanh):
  chot = mang[viTriCuoi]      // Chọn phần tử cuối làm chốt
  i = viTriDau - 1            // Biên vùng ≤ chốt
  FOR j FROM viTriDau TO viTriCuoi - 1:
    IF hamSoSanh(mang[j], chot) <= 0:
      i = i + 1
```

```
        SWAP(mang[i], mang[j])   // Đưa về vùng trái
    SWAP(mang[i+1], mang[viTriCuoi]) // Đặt chốt vào đúng vị trí
    RETURN i + 1                     // Trả về vị trí chốt
```

Ví dụ minh họa chi tiết:

Sắp xếp mảng sinh viên theo điểm trung bình tăng dần: [7.5, 9.0, 5.0, 8.0, 6.5]

Mảng ban đầu: [7.5, 9.0, 5.0, 8.0, 6.5]
Chọn chốt = 6.5 (phần tử cuối)

--- Phân hoạch ---
i = -1, j = 0: mang[0]=7.5 > 6.5 → không đổi
i = -1, j = 1: mang[1]=9.0 > 6.5 → không đổi
i = -1, j = 2: mang[2]=5.0 ≤ 6.5 → i=0, swap(mang[0],mang[2])
  → [5.0, 9.0, 7.5, 8.0, 6.5]
i = 0, j = 3: mang[3]=8.0 > 6.5 → không đổi
Đặt chốt: swap(mang[1], mang[4]) → [5.0, 6.5, 7.5, 8.0, 9.0]
Vị trí chốt = 1

--- Đệ quy nửa trái [5.0] → đã sắp xếp ---
--- Đệ quy nửa phải [7.5, 8.0, 9.0] → đã sắp xếp ---

Kết quả: [5.0, 6.5, 7.5, 8.0, 9.0] ✓

3.2.3. Hàm so sánh sinh viên

Hệ thống triển khai hàm tạo bộ so sánh (comparator factory) cho phép tạo hàm so sánh động theo nhiều tiêu chí khác nhau. Hàm taoHamSoSanh(truong, thuTu) nhận hai tham số: trường sắp xếp (id, name, gpa, class) và thứ tự (asc/desc), trả về hàm so sánh phù hợp.

Đặc biệt với tiêu chí sắp xếp theo tên (name), hệ thống sử dụng kỹ thuật tách phần tên cuối cùng trong họ tên đầy đủ. Ví dụ: "Nguyễn Văn An" → lấy "An" để so sánh, phù hợp với quy ước sắp xếp tên tiếng Việt. Hàm localeCompare() được sử dụng với locale "vi" để so sánh chính xác ký tự tiếng Việt.

3.3. Thuật toán sắp xếp BubbleSort

BubbleSort được triển khai trong hệ thống với mục đích minh họa và so sánh hiệu suất với QuickSort. Thuật toán duyệt mảng nhiều lượt, mỗi lượt so sánh hai phần tử liền kề và hoán đổi nếu sai thứ tự.

Mã giả (có tối ưu cờ dừng sớm):

```
FUNCTION bubbleSort(mang, hamSoSanh):
  n = mang.length
  FOR i FROM 0 TO n - 2:
    daDoi = false
    FOR j FROM 0 TO n - i - 2:
      IF hamSoSanh(mang[j], mang[j+1]) > 0:
        SWAP(mang[j], mang[j+1])
        daDoi = true
    IF daDoi == false:   // Không có hoán đổi → đã sắp xếp
      BREAK              // Dừng sớm!
```

Ví dụ minh họa:

Sắp xếp điểm TB: [7.5, 5.0, 9.0, 6.5]

Lượt 1:

So sánh 7.5 và 5.0 → swap → [5.0, 7.5, 9.0, 6.5]

So sánh 7.5 và 9.0 → OK

So sánh 9.0 và 6.5 → swap → [5.0, 7.5, 6.5, 9.0]

→ 9.0 "nổi" lên cuối

Lượt 2:

So sánh 5.0 và 7.5 → OK

So sánh 7.5 và 6.5 → swap → [5.0, 6.5, 7.5, 9.0]
→ 7.5 đúng vị trí

Lượt 3:
So sánh 5.0 và 6.5 → OK
→ Không có hoán đổi → DỪNG SỚM!

Kết quả: [5.0, 6.5, 7.5, 9.0]

Nhận xét: BubbleSort có $O(n^2)$ trung bình, chậm hơn QuickSort O(n log n) đáng kể khi n lớn. Với n = 1000 sinh viên: BubbleSort cần khoảng 1.000.000 phép so sánh, QuickSort chỉ cần khoảng 10.000 phép. Tuy nhiên, BubbleSort có ưu điểm đơn giản và ổn định (stable sort).

3.4. Thuật toán tìm kiếm tuần tự (Linear Search)

Tìm kiếm tuần tự là thuật toán tìm kiếm mặc định trong hệ thống, được sử dụng cho nhiều chức năng: kiểm tra trùng mã sinh viên khi thêm mới, tìm sinh viên theo mã để cập nhật/xóa, và tìm kiếm theo từ khóa trên nhiều trường.

3.4.1. Tìm kiếm đơn giản

Mã giả:

```
FUNCTION timTuanTu(mang, dieuKien):
   FOR i FROM 0 TO mang.length - 1:
      IF dieuKien(mang[i]) == true:
         RETURN i          // Tìm thấy → trả về vị trí
   RETURN -1                // Không tìm thấy
```

Ví dụ minh họa:

Tìm sinh viên có mã "SV003" trong mảng [SV001, SV002, SV003, SV004, SV005]:

Bước 1: i=0, kiểm tra SV001.id == "SV003"? → KHÔNG
Bước 2: i=1, kiểm tra SV002.id == "SV003"? → KHÔNG
Bước 3: i=2, kiểm tra SV003.id == "SV003"? → CÓ!

→ Trả về vị trí 2

→ Số bước: 3 (trong tổng 5 phần tử)

3.4.2. Tìm kiếm đa tiêu chí

Hệ thống mở rộng tìm kiếm tuần tự thành tìm kiếm đa tiêu chí: với mỗi sinh viên, kiểm tra xem từ khóa có xuất hiện trong mã sinh viên, họ tên, hoặc tên lớp. Thuật toán chuẩn hóa từ khóa (chuyển về chữ thường, xóa khoảng trắng thừa) rồi sử dụng hàm includes() để kiểm tra chuỗi con.

Mã giả:

```
FUNCTION timKiemSinhVien(danhSach, tuKhoa):
   tuKhoaChuanHoa = tuKhoa.toLowerCase().trim()
   IF tuKhoaChuanHoa rỗng: RETURN danhSach
   ketQua = []
   FOR mỗi sv TRONG danhSach:
      IF sv.id    CHỨA tuKhoaChuanHoa
      OR sv.name  CHỨA tuKhoaChuanHoa
      OR sv.class CHỨA tuKhoaChuanHoa
      OR sv.rank  CHỨA tuKhoaChuanHoa:
         ketQua.THÊM(sv)
   RETURN ketQua
```

Ví dụ minh họa:

Từ khóa: "nguyễn" → Chuẩn hóa: "nguyễn"

Duyệt danh sách 30 sinh viên:

- SV001 "Nguyễn Văn An" → name chứa "nguyễn" → ĐẠT
- SV002 "Trần Thị Bình" → không trùng → BỎ QUA
- ...
- SV014 "Nguyễn Thị Hồng" → name chứa "nguyễn" → ĐẠT
- SV026 "Nguyễn Thị Thảo" → name chứa "nguyễn" → ĐẠT

Kết quả: 3 sinh viên phù hợp

3.5. Thuật toán tìm kiếm nhị phân (Binary Search)

Tìm kiếm nhị phân được triển khai trong hệ thống như một phương pháp tìm kiếm hiệu quả cao cho trường hợp mảng đã được sắp xếp. Thuật toán hoạt động bằng cách liên tục chia đôi không gian tìm kiếm, mỗi bước loại bỏ một nửa số phần tử không cần xét.

Mã giả:

```
FUNCTION timNhiPhan(mang, giaTri, hamSoSanh):
   trai = 0
   phai = mang.length - 1
   WHILE trai <= phai:
      giua = trai + (phai - trai) / 2    // Tránh tràn số
      ketQua = hamSoSanh(mang[giua], giaTri)
      IF ketQua == 0:
         RETURN giua                    // Tìm thấy!
      ELSE IF ketQua < 0:
         trai = giua + 1                // Tìm nửa PHẢI
      ELSE:
         phai = giua - 1                // Tìm nửa TRÁI
   RETURN -1                            // Không tìm thấy
```

Ví dụ minh họa chi tiết:

Tìm sinh viên có điểm TB = 8.0 trong mảng ĐÃ SẮP XẾP theo điểm TB tăng dần:

Mảng: [4.5, 5.0, 6.0, 6.5, 7.0, 7.5, 8.0, 8.5, 9.0, 9.5]
Chỉ số: [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]
Tìm giá trị: 8.0

Bước 1: trai=0, phai=9, giua=(0+9)/2=4
  mang[4] = 7.0 < 8.0 → tìm nửa phải: trai = 5

Bước 2: trai=5, phai=9, giua=(5+9)/2=7

mang[7] = 8.5 > 8.0 → tìm nửa trái: phai = 6

Bước 3: trai=5, phai=6, giua=(5+6)/2=5

mang[5] = 7.5 < 8.0 → tìm nửa phải: trai = 6

Bước 4: trai=6, phai=6, giua=(6+6)/2=6

mang[6] = 8.0 == 8.0 → TÌM THẤY tại vị trí 6!

→ Chỉ cần 4 bước (thay vì 7 bước nếu dùng tìm kiếm tuần tự)

→ Với 10 phần tử: $\log_2(10) \approx 3.3$, tối đa 4 bước ✓

Lưu ý: Công thức tính chỉ số giữa sử dụng giua = trai + (phai - trai) / 2 thay vì giua = (trai + phai) / 2 để tránh tràn số khi trai + phai vượt quá giá trị tối đa của kiểu số. Đây là kỹ thuật lập trình an toàn được khuyến nghị.

3.6. Thuật toán thống kê Single-pass

Thuật toán thống kê tổng hợp của hệ thống là một minh chứng xuất sắc cho kỹ thuật "duyệt một lần" (single-pass). Thay vì duyệt mảng nhiều lần để thu thập từng chỉ số riêng, thuật toán chỉ duyệt MỘT LẦN DUY NHẤT nhưng cập nhật đồng thời tất cả các biến thống kê.

Mã giả:

```
FUNCTION thongKeTongHop(danhSach):
    // Khởi tạo
    tongDiem = 0
    diemMax = -∞, diemMin = +∞
    phanLoai = {xuatSac:0, gioi:0, kha:0, trungBinh:0, yeu:0}
    theoLop = {}, theoGioiTinh = {}


    // DUYỆT MỘT LẦN DUY NHẤT
    FOR mỗi sv TRONG danhSach:
        diemTB = sv.tinhDiemTB()
```

```
    tongDiem += diemTB
    IF diemTB > diemMax: diemMax = diemTB
    IF diemTB < diemMin: diemMin = diemTB
    // Phân loại học lực
    IF diemTB >= 9.0: phanLoai.xuatSac++
    ELSE IF diemTB >= 8.0: phanLoai.gioi++
    ELSE IF diemTB >= 6.5: phanLoai.kha++
    ELSE IF diemTB >= 5.0: phanLoai.trungBinh++
    ELSE: phanLoai.yeu++
    // Đếm theo lớp
    theoLop[sv.lop]++
    // Đếm theo giới tính
    theoGioiTinh[sv.gioiTinh]++

  diemTBChung = tongDiem / danhSach.length
  RETURN {diemTBChung, diemMax, diemMin, phanLoai, theoLop, theoGioiTinh}
```

Ví dụ minh họa:

Cho danh sách 5 sinh viên với điểm TB lần lượt: 9.2, 8.0, 6.5, 4.5, 7.5

Duyệt:

SV1 (9.2): tongDiem=9.2, max=9.2, min=9.2, xuatSac=1

SV2 (8.0): tongDiem=17.2, max=9.2, min=8.0, gioi=1

SV3 (6.5): tongDiem=23.7, max=9.2, min=6.5, kha=1

SV4 (4.5): tongDiem=28.2, max=9.2, min=4.5, yeu=1

SV5 (7.5): tongDiem=35.7, max=9.2, min=4.5, kha=2

Kết quả:

Điểm TB chung = 35.7 / 5 = 7.14

Điểm cao nhất = 9.2

Điểm thấp nhất = 4.5

Phân loại: Xuất sắc=1, Giỏi=1, Khá=2, TB=0, Yếu=1

→ Tất cả chỉ với 1 lần duyệt (5 bước) thay vì 5 lần (25 bước)!

Ngoài ra, thuật toán đếm phân nhóm sử dụng cấu trúc bảng băm (hash map) thông qua object JavaScript. Với mỗi sinh viên, lấy tên lớp (hoặc xếp loại, giới tính) làm khóa, tăng bộ đếm tương ứng lên 1. Chi phí: O(n) thời gian, O(k) bộ nhớ phụ (k = số nhóm, rất nhỏ).

3.7. Thuật toán phân loại học lực

Thuật toán phân loại học lực sử dụng cấu trúc rẽ nhánh if-else với các ngưỡng (threshold) được sắp xếp giảm dần. Việc sắp xếp giảm dần giúp tối ưu: sinh viên xuất sắc chỉ cần 1 phép so sánh, sinh viên yếu cần 4 phép so sánh, nhưng trung bình thì chỉ cần khoảng 2-3 phép so sánh.

Mã giả:

```
FUNCTION xepLoaiHocLuc(diemTB):
  IF diemTB >= 9.0: RETURN "Xuất sắc"
  IF diemTB >= 8.0: RETURN "Giỏi"
  IF diemTB >= 6.5: RETURN "Khá"
  IF diemTB >= 5.0: RETURN "Trung bình"
  RETURN "Yếu"
```

Ví dụ minh họa:

Sinh viên	Điểm TB	Xếp loại	Số phép so sánh
Nguyễn Văn An	9.17	Xuất sắc	1
Trần Thị Bình	8.50	Giỏi	2
Lê Hoàng Cường	7.50	Khá	3
Hoàng Văn Em	6.00	Trung bình	4
Đỗ Minh Quang	4.83	Yếu	5 (tất cả)

Hình 3.7.1: Thuật toán phân loại học lực

Độ phức tạp: O(1) — tối đa 5 phép so sánh, không phụ thuộc vào kích thước dữ liệu.

Tóm tắt chương 3

Chương 3 trình bày các thuật toán được sử dụng trong chương trình. Bao gồm các thao tác của mảng động như thêm, xóa, chèn và mở rộng mảng. Thuật toán QuickSort được dùng để sắp xếp chính vì nhanh và hiệu quả, còn BubbleSort dùng để minh họa so sánh. Tìm kiếm tuần tự được dùng khi dữ liệu chưa sắp xếp, còn tìm kiếm nhị phân dùng khi dữ liệu đã sắp xếp. Ngoài ra còn có thuật toán thống kê duyệt một lần và thuật toán xếp loại học lực. Chương này thể hiện rõ cách áp dụng lý thuyết vào xây dựng phần mềm quản lý sinh viên hoàn chỉnh.

CHƯƠNG 4: KẾT QUẢ VÀ BÀN LUẬN

4.1. Môi trường và công cụ phát triển

Để hiện thực hóa phần mềm quản lý sinh viên, bài tập lớn đã sử dụng các công nghệ và thư viện cụ thể như sau:

- **JavaScript thuần (Vanilla JS):** Đóng vai trò xử lý toàn bộ logic nghiệp vụ, xây dựng cấu trúc dữ liệu mảng động và triển khai các thuật toán (sắp xếp, tìm kiếm, thống kê) mà không phụ thuộc vào các framework bên thứ ba.
- **HTML5 & CSS3:** Xây dựng bộ khung giao diện theo mô hình trang web đơn (Single Page Application - SPA) và thiết kế bố cục trực quan, hiện đại.
- **Thư viện Chart.js:** Sử dụng để vẽ các biểu đồ thống kê trực quan (biểu đồ tròn phân bố học lực, biểu đồ cột thống kê theo lớp) dựa trên dữ liệu thực tế thu thập được.
- **Thư viện FontAwesome 6.4.0:** Cung cấp hệ thống biểu tượng (icon) vector sắc nét, giúp nhận diện các nút chức năng (thêm, sửa, xóa, tìm kiếm) trên giao diện một cách thân thiện, dễ nhìn.
- **Web Storage (localStorage):** Lưu trữ danh sách sinh viên cục bộ trên trình duyệt dưới định dạng JSON, đảm bảo dữ liệu được lưu trữ bền vững và không bị mất đi khi người dùng đóng hoặc tải lại trang web.

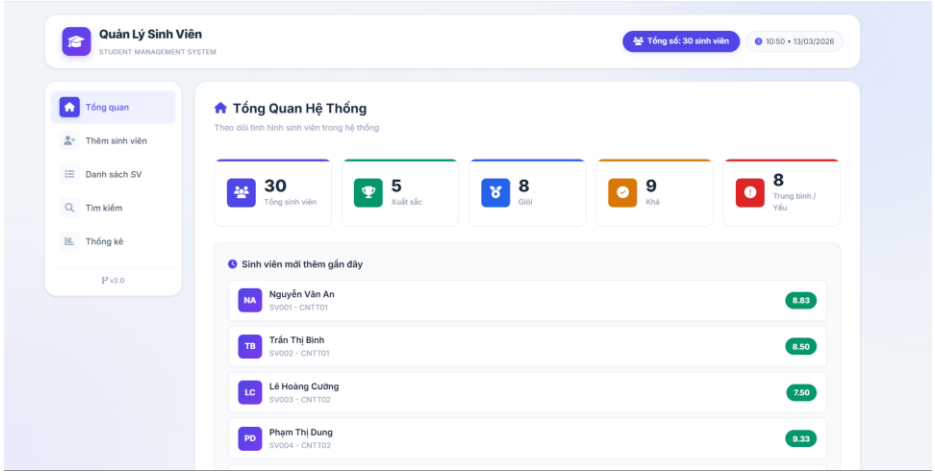
4.2. Giao diện và kết quả thực thi các chức năng

Hệ thống được thiết kế dưới dạng trang web đơn (Single Page Application - SPA), mang lại trải nghiệm mượt mà, cho phép người dùng chuyển đổi giữa các chức năng nhanh chóng mà không cần tải lại trang. Các phân hệ chức năng chính được cài đặt và cho ra kết quả thực thi cụ thể như sau:

4.2.1. Phân hệ Tổng quan (Dashboard)

Giao diện Tổng quan là màn hình đầu tiên hiển thị khi khởi động ứng dụng. Phân hệ này cung cấp cái nhìn tổng quát về tình trạng dữ liệu hiện tại thông qua các thẻ thống kê trực quan (Card). Các thông số hiển thị bao gồm: tổng số sinh viên trong hệ thống và số lượng sinh viên phân theo từng mức học lực (Xuất sắc, Giỏi, Khá, Trung bình/Yếu).

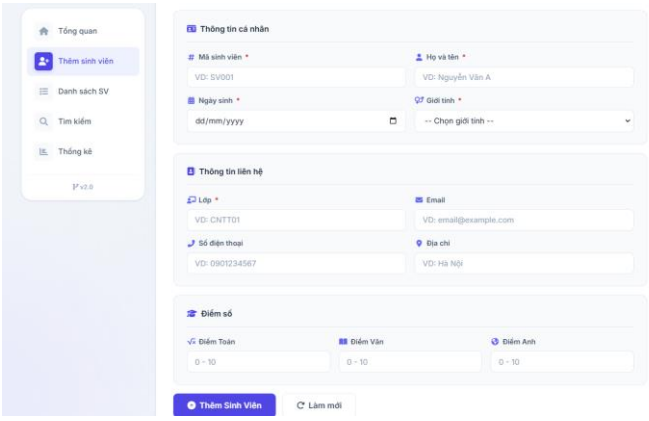
Phía dưới là khu vực hiển thị danh sách các sinh viên mới được thêm vào hệ thống gần đây nhất, giúp người dùng dễ dàng theo dõi tiến độ cập nhật dữ liệu hàng ngày.



Hình 4.2.1: Giao diện Tổng quan của hệ thống.

4.2.2. Phân hệ Thêm mới sinh viên

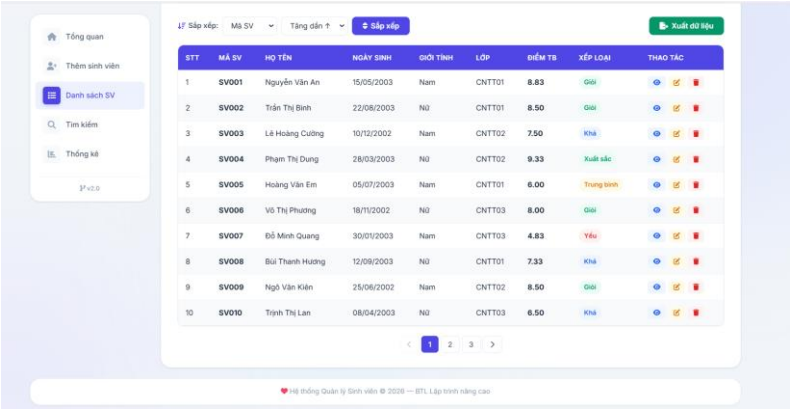
Phân hệ này cung cấp một biểu mẫu (form) nhập liệu được thiết kế rõ ràng, chia thành ba nhóm thông tin chính: Thông tin cá nhân, Thông tin liên hệ và Điểm số. Trong quá trình người dùng nhập liệu, hệ thống áp dụng thuật toán kiểm tra tính hợp lệ (Validation) theo thời gian thực. Các ràng buộc được thiết lập chặt chẽ như: mã sinh viên không được để trống và không được trùng lặp (kiểm tra bằng thuật toán tìm kiếm tuần tự), ngày sinh không vượt quá thời gian hiện tại, và điểm số các môn bắt buộc nằm trong khoảng từ 0 đến 10. Khi dữ liệu hợp lệ, đối tượng sinh viên mới sẽ được khởi tạo và đưa vào cấu trúc mảng động thông qua thao tác thêm vào cuối mảng với chi phí thời gian trung bình O(1).



Hình 4.2.2: Giao diện biểu mẫu thêm mới sinh viên và tính năng kiểm tra dữ liệu.

4.2.3. Phân hệ Quản lý danh sách sinh viên

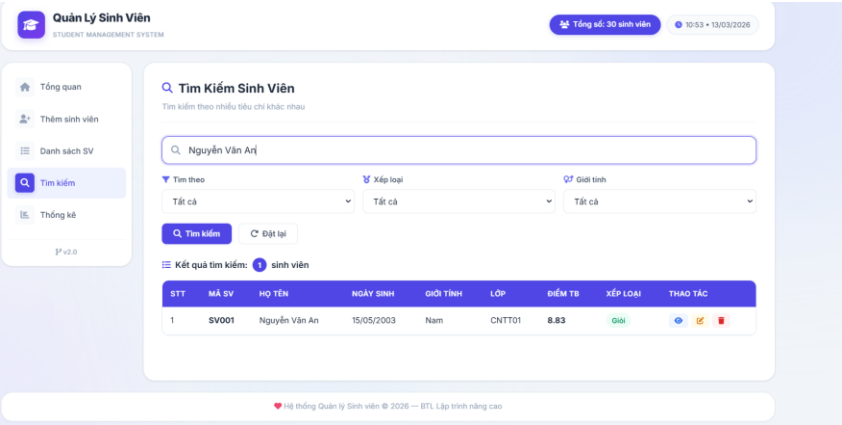
Dữ liệu sinh viên được xuất ra và hiển thị dưới dạng bảng chi tiết. Nhằm xử lý trường hợp số lượng sinh viên lớn, bảng được tích hợp tính năng phân trang (pagination) giúp giao diện luôn gọn gàng và không bị quá tải. Điểm nổi bật của phân hệ này là việc tích hợp công cụ sắp xếp động dựa trên thuật toán QuickSort. Người dùng có thể tùy chọn sắp xếp danh sách sinh viên theo nhiều tiêu chí như: Mã sinh viên, Họ tên, Điểm trung bình hoặc Lớp học, theo thứ tự tăng dần hoặc giảm dần. Ngay sau khi chọn, thuật toán sẽ xử lý và bảng dữ liệu được cập nhật kết quả sắp xếp ngay lập tức.



Hình 4.2.3: Giao diện quản lý danh sách sinh viên và công cụ sắp xếp động.

4.2.4. Phân hệ Tìm kiếm

Chức năng tìm kiếm được thiết kế linh hoạt với khả năng tìm kiếm đa tiêu chí. Người dùng có thể nhập từ khóa tự do để tìm kiếm theo mã sinh viên, họ tên hoặc lớp. Bên cạnh đó, hệ thống cung cấp các bộ lọc thả xuống nâng cao như lọc theo xếp loại học lực và giới tính. Khi thực hiện tìm kiếm, thuật toán tìm kiếm tuần tự kết hợp so sánh chuỗi sẽ duyệt qua mảng động, trích xuất các kết quả khớp với điều kiện và hiển thị trực quan trên một bảng kết quả riêng biệt.



Hình 4.2.4: Kết quả thực thi chức năng tìm kiếm đa tiêu chí.

4.2.5. Phân hệ Thống kê

Phân hệ thống kê thể hiện rõ nhất hiệu suất của thuật toán "duyệt một lần" (Single-pass) được cài đặt trong bài tập lớn. Chỉ với một vòng lặp duyệt qua mảng động, hệ thống trích xuất và tính toán đồng thời thành công các chỉ số: điểm trung bình chung toàn trường, điểm cao nhất và điểm thấp nhất. Các kết quả thống kê này được trực quan hóa sinh động nhờ thư viện Chart.js, bao gồm biểu đồ tròn thể hiện tỷ lệ phân bố xếp loại học lực và biểu đồ cột thống kê số lượng sinh viên theo từng lớp. Phía dưới cùng là bảng tổng hợp chi tiết các số liệu để người dùng tiện đối chiếu.

Hình 4.2.5: Giao diện thống kê tổng hợp và các biểu đồ trực quan.

Tóm tắt chương 4

Chương 4 trình bày kết quả cài đặt và đánh giá phần mềm quản lý sinh viên sử dụng mảng động. Ứng dụng được xây dựng thành công bằng JavaScript thuần, HTML/CSS và Chart.js. Các phân hệ chức năng (thêm mới, quản lý danh sách, tìm kiếm đa tiêu chí, thống kê trực quan) đều hoạt động mượt mà, phản hồi nhanh. Kết quả thực thi thực tế khẳng định các thuật toán cốt lõi như QuickSort, Binary Search và thống kê Single-pass đã phát huy tốt hiệu năng tối ưu bộ nhớ và thời gian xử lý. Tuy nhiên, hệ thống vẫn còn hạn chế do sử dụng lưu trữ cục bộ (localStorage). Từ đó, báo cáo đề xuất hướng phát triển tích hợp máy chủ (Backend) và cơ sở dữ liệu tập trung để phần mềm có thể ứng dụng rộng rãi hơn trong thực tế.

KẾT LUẬN

Bài tập lớn đã hoàn thành việc xây dựng một hệ thống quản lý sinh viên hoàn chỉnh sử dụng cấu trúc dữ liệu mảng động và giao diện đồ họa web. Hệ thống được phát triển bằng JavaScript thuần với kiến trúc module hóa rõ ràng, bao gồm 9 file JavaScript phân chia theo chức năng.

Về mặt cấu trúc dữ liệu, hệ thống triển khai thành công lớp DynamicArray (Mảng động) tự viết với đầy đủ các thao tác: push, pop, get, set, insertAt, removeAt, filter, sort, binarySearch, clone. Cơ chế resize gấp đôi dung lượng đảm bảo thao tác thêm phần tử đạt O(1) trung bình theo phân tích khấu hao.

Về mặt thuật toán, hệ thống áp dụng đa dạng các thuật toán kinh điển: QuickSort O(n log n) cho sắp xếp, BubbleSort $O(n^2)$ cho minh họa, tìm kiếm tuần tự O(n) cho dữ liệu chưa sắp xếp, tìm kiếm nhị phân O(log n) cho dữ liệu đã sắp xếp, thống kê single-pass O(n) cho hiệu quả cao, và thuật toán phân loại dựa trên ngưỡng O(1) cho xếp loại học lực.

Về mặt chức năng, hệ thống đáp ứng đầy đủ các nghiệp vụ quản lý sinh viên: thêm mới với validate, cập nhật, xóa, hiển thị danh sách phân trang, tìm kiếm đa tiêu chí, sắp xếp đa trường, thống kê tổng hợp với biểu đồ trực quan, xuất/nhập dữ liệu JSON, và lưu trữ bền vững qua localStorage. Giao diện được thiết kế hiện đại, trực quan với HTML/CSS responsive.

HƯỚNG PHÁT TRIỂN

Để có thể đưa phần mềm vào ứng dụng trong thực tiễn với quy mô lớn hơn, hệ thống cần được nghiên cứu và phát triển thêm theo các hướng sau:

- Tích hợp hệ thống máy chủ (Backend) và cơ sở dữ liệu quan hệ (như MySQL hoặc SQL Server) để quản lý dữ liệu tập trung, đảm bảo tính toàn vẹn và an toàn thông tin.
- Bổ sung cơ chế phân quyền người dùng (Quản trị viên, Giảng viên, Sinh viên) đi kèm với tính năng đăng nhập bảo mật.
- Mở rộng thêm các module nghiệp vụ như quản lý học phí, quản lý môn học động và hỗ trợ xuất các báo cáo thống kê định kỳ ra tệp tin định dạng Excel hoặc PDF.

TÀI LIỆU THAM KHẢO

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein (2009), *Introduction to Algorithms (3rd Edition)*, MIT Press. (Tài liệu tham khảo nền tảng về thuật toán QuickSort, tìm kiếm nhị phân và phân tích độ phức tạp Big-O).

[2] Mozilla Developer Network (MDN), *JavaScript Reference & Data Structures*, truy cập tại: https://developer.mozilla.org/ (Tài liệu tham khảo chuẩn về cú pháp JavaScript, Web Storage API và xử lý chuỗi/mảng).

[3] Chart.js Contributors, *Chart.js Documentation*, truy cập tại: https://www.chartjs.org/docs/latest/ (Tài liệu tham khảo cách vẽ và cấu hình biểu đồ trực quan cho phân hệ thống kê).

[4] Fonticons, Inc., *FontAwesome 6.4.0 Documentation*, truy cập tại: https://fontawesome.com/ (Sử dụng bộ biểu tượng giao diện đồ họa).